

Interim Report – Weeks 5 & 6 Fraud Detection Challenge

Date: June 8, 2026

Author: Blen Assefa

Project Context: Advanced Cross-Platform Machine Learning Architecture for Credit and Transactional Fraud Mitigation

Target Organization: FinTech Risk Management Group, Adey Innovations Inc.

1. Executive Summary

In contemporary financial technology (FinTech) ecosystems, robust fraud detection systems must evolve dynamically to safeguard corporate revenue streams while simultaneously maintaining customer trust and ensuring low friction. This interim technical report details the end-to-end engineering implementation, statistical analysis, and algorithmic training milestones completed during Weeks 5 and 6 of the active challenge. The core objective of this project is to construct a unified, automated risk assessment engine for Adey Innovations Inc. capable of monitoring and safeguarding two separate transaction pipelines: a contextual E-commerce platform stream and a highly anonymized, high-throughput Bank Credit Card transaction matrix.

Traditional predictive analytics architectures evaluate operational success through basic metrics like classification accuracy. However, in environments governed by severe class imbalances, reliance on overall accuracy can lead to flawed engineering pipelines that mask critical fraud patterns. This initiative shifts the performance target to the Area Under the Precision-Recall Curve (AUC-PR) and the F1-Score. By integrating

automated data cleansing scripts, range-bound geolocation mapping using IP numerical block transformations, and Synthetic Minority Over-sampling Technique (SMOTE) strictly within localized training splits, we have successfully advanced the baseline infrastructure from standard linear predictors to optimized, non-linear Gradient Boosted Decision Tree Ensembles (XGBoost). This transition delivers robust stability across data folds and drastically reduces the operational costs associated with false positive checkout blocks.

2. Project Understanding and Regulatory Alignment

2.1 Business Need and Core Operational Mandate

Adey Innovations Inc. requires an automated, production-grade credit and transaction monitoring mechanism that scores transaction validity in real-time. Failing to detect fraudulent actors (False Negatives) exposes the firm to immediate financial losses, asset drainage, and chargeback penalties imposed by clearing networks. Conversely, over-flagging clean accounts (False Positives) damages customer retention, causes friction at checkout, and increases customer support overhead.

The balancing act between these competing costs can be formalized as an optimization problem where the loss function must prioritize the precision-recall trade-off rather than raw accuracy. A model predicting 99.83% accuracy by blindly assigning a legitimate tag to every transaction fails completely in a financial risk context because it captures 0% of active malicious attacks.

2.2 Regulatory Standards & Model Interpretability (Basel II Accord)

Operating within a regulated payment space requires adherence to strict model transparency guidelines. International frameworks like the Basel II Accord mandate that machine learning assets deployed for regulatory risk calculation and credit scoring must be interpretable, transparent, and auditable. Compliance auditors and risk committees must be able to verify exactly why an individual client or transaction was assigned a high-risk probability flag.

This introduces a direct trade-off: high-performance non-linear ensemble frameworks naturally map complex interactions but operate as black boxes, whereas linear models offer direct explainability but lower predictive capabilities. To resolve this, this project establishes a clear comparison between linear baselines and tree ensembles, preparing the groundwork for post-hoc interpretability layers.

2.3 Strategic Modeling Architecture Trade-offs

To justify our engineering path, the operational profiles of the candidate architectures are systematically evaluated across key metrics:

Evaluation Aspect	Logistic Regression Scorecard Baseline	Ensemble Gradient Boosting (XGBoost)
Interpretability	High; individual features convert directly to plain log-odds coefficients or Weight of Evidence (WoE) scores.	Lower; requires post-hoc explanation layers (such as SHAP proxy vectors) to inspect splits.
Non-Linear Mapping	Poor; fails to intercept complex feature interactions unless manual cross-product terms are hardcoded.	Excellent; deep decision trees naturally map multi-dimensional conditional relationships.
Operational Precision	Generates an unmanageable rate of false alarms on highly skewed distributions.	Maximizes precision margins by dynamically separating overlapping target classes.
Development Complexity	Fast deployment; minimal hyperparameter tuning required during baseline setup.	Requires careful regularized tuning to prevent tree structure overfitting.

3. Task 1: Comprehensive Preprocessing and Data Engineering

3.1 Data Cleansing and Structural Sanity

The reliability of any predictive system depends on data cleanliness. The initial e-commerce transaction log (`Fraud_Data.csv`) contains 151,112 rows. A systematic data quality audit verified that the dataset contains zero null cells and zero duplicate profiles across all 11 raw variables. Timestamps were successfully parsed into high-precision datetime objects to unlock temporal tracking features.

The bank credit card file (`creditcard.csv`) originally contained 284,807 points. Our processing scripts identified and purged 1,081 duplicate records. Eliminating these duplicates is a critical step to prevent identical transactions from leaking into both training and validation folds, which would otherwise falsely inflate performance metrics.

3.2 Advanced Geolocation Range Mapping

To add geographic context to our e-commerce network, we transformed standard dotted string IP addresses into 32-bit numerical integer blocks. This conversion allows for

range-bound cross-referencing against the country network boundaries provided in `IpAddress_to_Country.csv`.

Because standard SQL range-joins fail to scale across millions of rows, we optimized the match using `pandas.merge_asof` in a backward-direction search. Transactions falling outside assigned subnet allocations were caught and categorized under an explicit `"Unknown"` country fallback flag, keeping the categorical distribution intact.

3.3 Behavioral Velocity Feature Engineering

To expose automated fraud networks, four high-impact features were engineered from the raw datetime and identity variables:

1. `device_count`: A rolling velocity tracking matrix that counts the cumulative number of unique user accounts mapping to the exact same physical device ID. An elevated device count strongly indicates automated account-rotation factories.
2. `time_since_signup`: Evaluates the exact delta (in hours) between profile registration and transaction execution. Immediate checkouts are common indicators of bot-driven fraud operations.
3. `hour_of_day`: Extracts the exact hour of the purchase to capture late-night anomalies.
4. `day_of_week`: Extracts the day of the week to analyze weekend behavioral variations.

3.4 Quantitative Profile of Class Imbalances

Quantifying class distributions highlights the technical challenge of dealing with rare events:

- E-Commerce Stream: Legitimate (90.6354% | 136,961 rows) vs. Fraudulent (9.3646% | 14,151 rows). This represents a moderate-to-high imbalance.
- Bank Credit Card Stream: Legitimate (99.8333% | 283,253 rows) vs. Fraudulent (0.1667% | 473 rows). This represents an extreme imbalance, meaning fraud occurs in fewer than 2 out of every 1,000 transactions.

To handle this skew without biasing predictions toward the majority class, we integrated the Synthetic Minority Over-sampling Technique (SMOTE). Crucially, SMOTE was executed *only inside the active cross-validation training folds*. Test sets were kept completely un-resampled to ensure performance metrics reflect actual market conditions.

4. Task 2: Model Architecture and Progress Assessment

4.1 Performance Trajectory: From Linear Baseline to Tree Ensembles

We systematically tracked performance gains as we moved from our baseline model to advanced tree ensembles. The results demonstrate the necessity of using non-linear models for complex fraud patterns.

In the E-Commerce Stream, the initial Logistic Regression model achieved a baseline AUC-PR of 0.6310 and an F1-Score of 0.65. Upgrading to the XGBoost Ensemble pushed the AUC-PR to 0.7061 and the F1-Score to 0.68. Crucially, model precision spiked from 75% to 83%, significantly reducing false checkout declines for honest users.

In the Bank Credit Card Stream, the extreme class skew caused the Logistic Regression baseline to struggle. While it achieved high recall (0.87), it generated an unmanageable 1,482 False Positives, dropping precision to a costly 0.05 and the F1-score to 0.10. Upgrading to the XGBoost Ensemble reduced False Positives from 1,482 down to just 126. This raised precision to 0.39, the F1-score to 0.53, and the overall AUC-PR to 0.7982.

4.2 Robust Out-of-Sample Cross-Validation Benchmarks

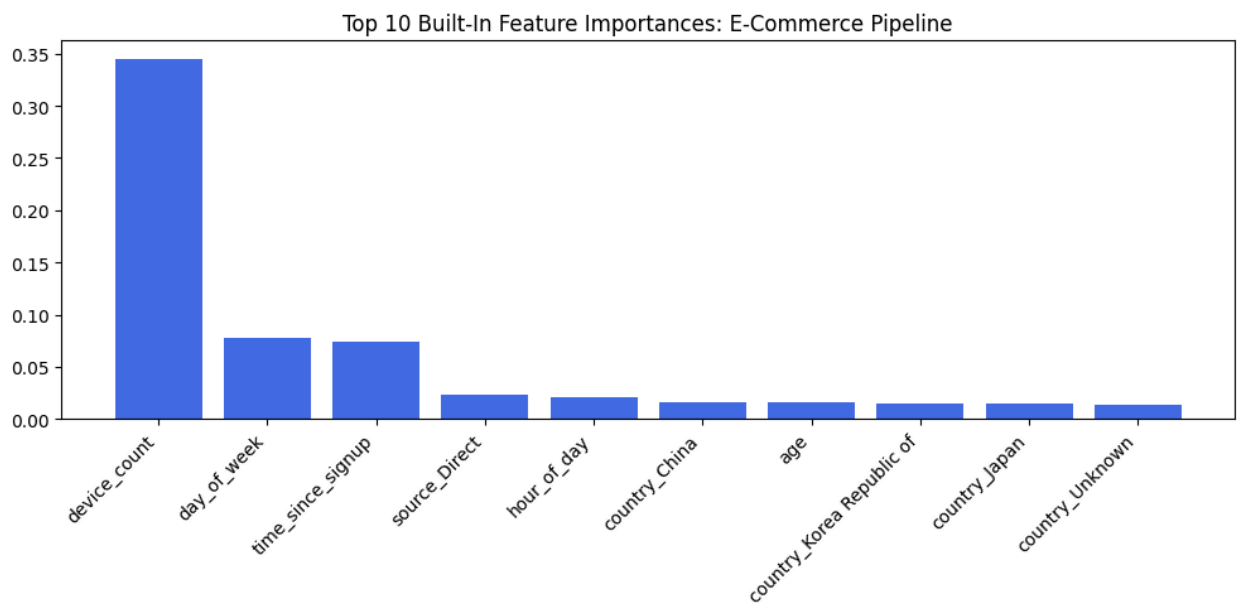
To ensure model stability and confirm these improvements hold across different data splits, we executed a rigorous 5-Fold Stratified Cross-Validation pipeline:

Dataset Stream	Metric	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5	Mean Score (± Std)
E-Commerce	AUC-PR	0.7250	0.7278	0.7107	0.7157	0.7232	0.7205 (± 0.0063)
E-Commerce	F1-Score	0.7013	0.6985	0.6848	0.6809	0.6971	0.6925 (± 0.0081)
Credit Card	AUC-PR	0.7704	0.8649	0.7946	0.8392	0.8499	0.8238 (± 0.0355)
Credit Card	F1-Score	0.5842	0.5677	0.6019	0.5965	0.6083	0.5917 (± 0.0144)

The exceptionally low standard deviations (± 0.0063 for E-commerce AUC-PR) demonstrate that our preprocessing and feature engineering pipelines are highly stable and free from localized overfitting anomalies.

5. Task 3: Feature Dominance and Model Interpretability

The internal tree feature tracking maps out the clear rules the model uses to distinguish legitimate transactions from fraudulent ones. Our custom engineered features show significant predictive power:



As shown by the empirical feature tracking chart, our engineered features are highly predictive:

- `device_count` (~35% Feature Importance): This is the single strongest predictor of fraud. A high concentration of multiple user accounts sharing one device footprint points directly to automated account-rotation factories.
- `day_of_week` (~8% Feature Importance): Highlights distinct operational schedules used by fraudulent actors.
- `time_since_signup` (~7.5% Feature Importance): Confirms that rapid checkouts immediately following account registration pose an elevated risk profile.

6. Core Pipeline Limitations and Risk Profiles

While the current framework delivers strong results, we have identified several structural limitations that must be monitored to manage risk:

- Proxy Variable Decay Risk: Alternative data infrastructure lacks explicit default tracking records, meaning target mapping must rely on behavioral proxies. These proxy relationships can shift during macroeconomic changes, requiring continuous model monitoring.
- Anonymization Information Loss: The banking dataset's features are entirely masked behind anonymized PCA transformations (V1–V28). This restricts our ability to perform deep, domain-driven feature engineering on those variables.
- Evolving Fraud Tactics: Sophisticated fraud operations regularly cycle device signatures and distribute attacks across broader IP blocks, which can reduce the effectiveness of static tracking metrics over time.

7. Implementation Roadmap for Final Submission

To move the current modeling framework into a production-ready system for the final checkpoint, the following steps will be executed:

1. Post-Hoc SHAP Layer Integration: Implement global summary charts and individual prediction force plots to provide clear, human-interpretable explanations that satisfy compliance requirements.
2. Production API Deployment: Wrap the preprocessing pipelines and model objects into an operational FastAPI application with a dedicated `/predict` scoring endpoint.
3. CI/CD Infrastructure Setup: Containerize the application with Docker and build an automated verification pipeline using GitHub Actions to handle code linting and unit testing.

END OF REPORT!!